

DOCUMENTAÇÃO TÉCNICA

Sistema de Consolidação e Consulta de Despesas de Obras

Processamento de relatórios PDF • PostgreSQL/Supabase • LLM multimodal • Text-to-SQL

Objetivo da documentação

Apresentar o problema de negócio, a solução efetivamente implementada, a arquitetura, os fluxos de execução, o uso de modelos LLM, a persistência e consulta dos dados, a geração de relatórios e os procedimentos de execução.

Documento técnico

Karine Versieux Magalhães

Curso BI Master PUC Rio

Matrícula: 231.100.986 Turmas (23.2 e 24.1)

Sumário

- [1. Visão geral do projeto](#)
- [2. Descrição do problema](#)
- [3. Objetivos](#)
 - [3.1 Objetivo geral](#)
 - [3.2 Objetivos específicos](#)
- [4. Solução desenvolvida](#)
- [5. Arquitetura da solução](#)
 - [5.1 Componentes](#)
- [6. Fluxo geral de execução](#)
- [7. Processamento dos relatórios PDF e uso do LLM multimodal](#)
 - [7.1 Entrada do LLM](#)
 - [7.2 Informações solicitadas](#)
 - [7.3 Saída estruturada](#)
- [8. Modelos estruturados e validação dos dados](#)
- [9. Banco de dados, catálogo e persistência](#)
 - [9.1 Construção automática do catálogo](#)
 - [9.2 Persistência](#)
- [10. Consultas em linguagem natural: Text-to-SQL](#)
 - [10.1 Configuração do modelo](#)
 - [10.2 Saída estruturada](#)
- [11. Validação e segurança do SQL](#)
- [12. Conciliação de despesas e comprovantes](#)
- [13. Geração do relatório consolidado](#)
- [14. Estrutura do código](#)
- [15. Exemplos de execução](#)
 - [15.1 Processamento de relatórios](#)
 - [15.2 Consulta em linguagem natural](#)
 - [15.3 Geração de relatório](#)
- [16. Configuração e execução do projeto](#)
 - [16.1 Variáveis relevantes](#)
- [17. Tecnologias utilizadas](#)
- [18. Comparação entre proposta e implementação](#)
- [19. Referências e links](#)
 - [Fontes e referências utilizadas no desenvolvimento e na documentação](#)

1. Visão geral do projeto

O projeto tem como finalidade apoiar a gestão e a análise de relatórios de despesas de obras, automatizando a extração de informações presentes em documentos PDF, a persistência estruturada dos dados, a conciliação entre despesas e comprovantes, a geração de relatórios consolidados e a realização de consultas ao banco de dados por meio de linguagem natural.

Princípio central da solução

O projeto combina LLMs em tarefas que exigem interpretação de documentos e linguagem natural com processos determinísticos em Python e PostgreSQL para validação, persistência, conciliação e geração de relatórios.

2. Descrição do problema

A motivação para o desenvolvimento da solução decorre da necessidade de conferir manualmente os valores das despesas, taxas de administração, a correspondência entre despesas e respectivos documentos comprobatórios, que normalmente são apresentados em relatórios posteriores ao lançamento das despesas. Além disso, identificou-se a necessidade de disponibilizar uma visão consolidada de todos os períodos.

Necessidade	Tratamento no sistema
Extrair informações dos PDFs	LLM multimodal + saída estruturada
Evitar duplicidade de relatórios	Consulta prévia ao banco
Restringir dados ao contrato	Validação do contrato informado pelo usuário
Associar despesas a comprovantes	Rotina de conciliação determinística
Consultar dados	Linguagem natural → SQL → validação → execução
Consolidar informações	Consultas ao PostgreSQL + ReportLab

3. Objetivos

3.1 Objetivo geral

Desenvolver uma solução computacional para automatizar etapas de consolidação, consulta e análise de dados de despesas de obras, reduzindo o esforço manual associado ao tratamento dos relatórios PDF.

3.2 Objetivos específicos

- Processar relatórios PDF contendo quadro-resumo e documentos comprobatórios de despesas.
- Extrair informações estruturadas dos documentos.
- Persistir os dados no banco PostgreSQL hospedado no Supabase.
- Evitar o reprocessamento de relatórios já cadastrados.
- Associar despesas aos comprovantes disponíveis.
- Gerar relatório consolidado em PDF.
- Permitir consultas ao banco por linguagem natural.
- Restringir a geração de SQL ao contexto do banco, às regras de negócio e ao contrato selecionado.

4. Solução desenvolvida

A implementação atual está organizada como uma aplicação modular Python. A função principal apresenta ao usuário três operações de negócio: processamento de relatórios, geração de relatório consolidado e consulta de dados em linguagem natural.

Opção	Operação	Resultado
1	Processar relatório(s) de despesas	Dados extraídos, validados, armazenados e conciliados
2	Gerar relatório consolidado	Arquivo PDF com tabelas e visualizações
3	Consultar dados	SQL gerado, validado e executado; resultado em tabela
0	Sair	Encerramento da execução

A execução é orientada por um código de obra/contrato. O sistema valida esse código no banco, obtém o cliente associado e utiliza o identificador do contrato nas operações subsequentes.

5. Arquitetura da solução

A arquitetura efetivamente implementada é modular. Cada módulo possui uma responsabilidade específica, enquanto config.py centraliza credenciais, parâmetros, modelos LLM, regras de negócio e exemplos de consulta.

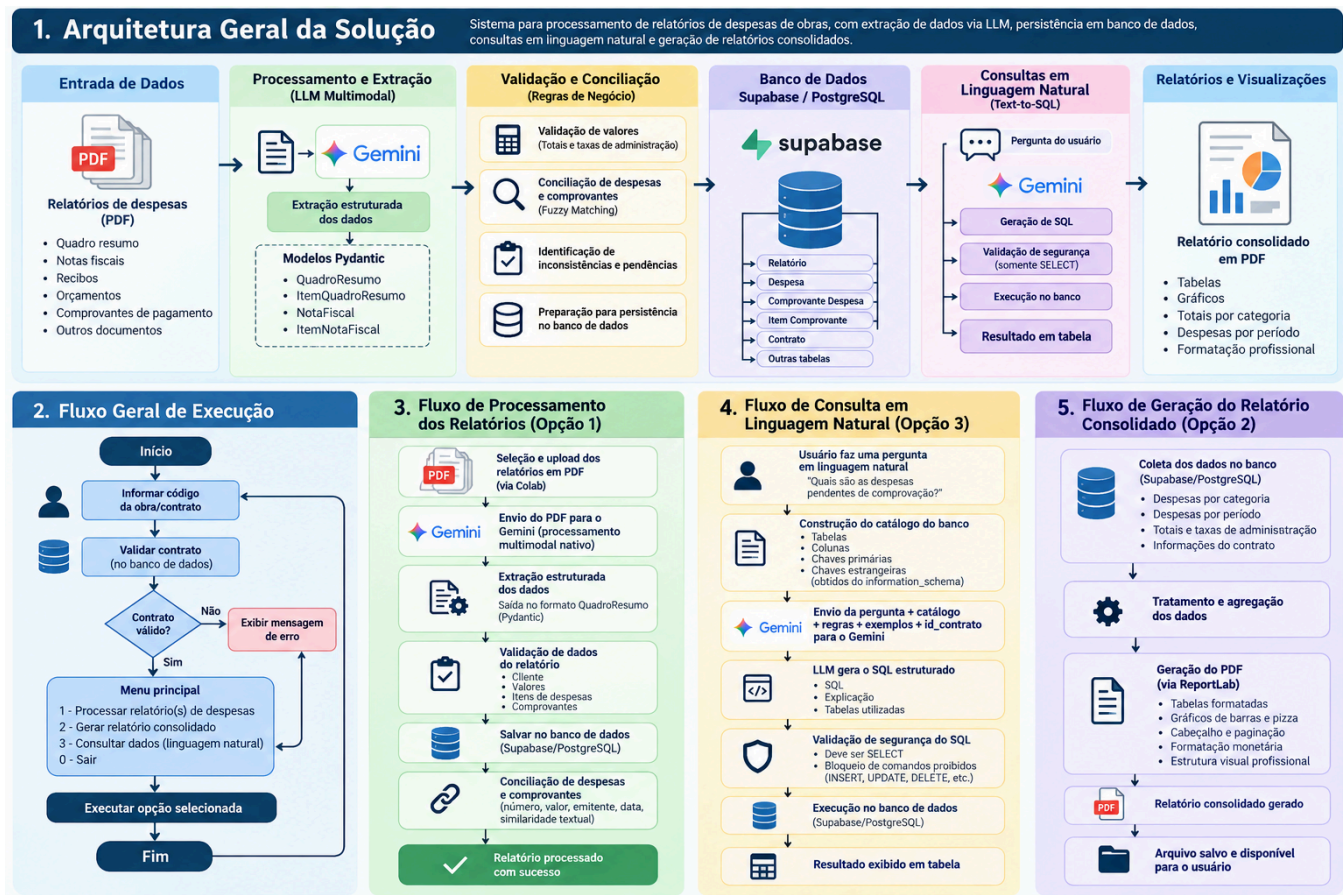


Figura 1 — Arquitetura e principais fluxos do sistema.

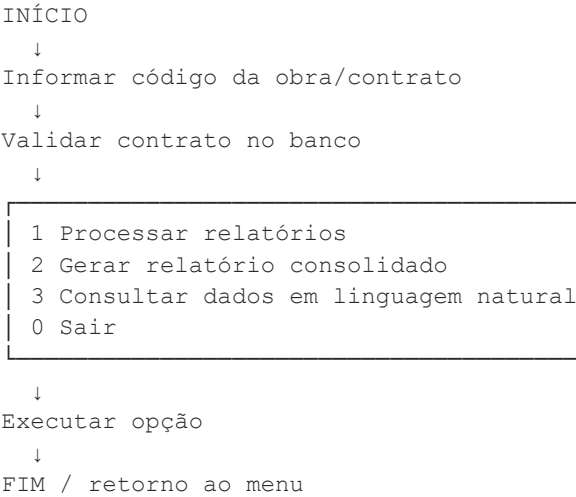
5.1 Componentes

Módulo	Responsabilidade
main.py	Interface principal, seleção do contrato e orquestração das três opções de uso.
config.py	Variáveis de ambiente, credenciais, modelos LLM, regras de negócio e exemplos.
processing.py	Processamento dos PDFs, normalização de textos e datas e extração via Gemini.
models.py	Modelos Pydantic para representar dados extraídos e respostas estruturadas do LLM.

database.py	Conexão, catálogo do banco, persistência, validações e conciliação.
consultas.py	Geração de SQL via Gemini, validação e execução no PostgreSQL.
report.py	Construção do relatório consolidado em PDF e geração de gráficos.

6. Fluxo geral de execução

A aplicação inicia solicitando o código da obra/contrato. Após a validação, o usuário acessa o menu principal. A partir desse ponto, cada opção conduz a um fluxo específico.



Controle por contrato

A seleção do contrato ocorre antes das operações de negócio. O identificador do contrato é utilizado para restringir o processamento e as consultas aos dados correspondentes.

7. Processamento dos relatórios PDF e uso do LLM multimodal

O primeiro uso de LLM ocorre na extração de dados dos relatórios PDF. O módulo processing.py utiliza o SDK google-genai, envia o PDF ao serviço Gemini e solicita a análise do documento completo.

7.1 Entrada do LLM

- Arquivo PDF do relatório.
- Prompt especializado para extração de despesas de obras.
- Esquema estruturado de saída baseado em Pydantic.

7.2 Informações solicitadas

- Cliente, endereço, número e data do relatório.
- Itens do quadro-resumo.
- Valores e datas de despesas.
- Identificação de notas fiscais, recibos e outros comprovantes.
- Emitentes, remetentes e valores.
- Itens detalhados das notas fiscais.
- Classificação dos comprovantes e categorias de insumos.
- Valores relacionados à administração de materiais e serviços.

7.3 Saída estruturada

A resposta não é tratada como texto livre. O sistema solicita JSON estruturado e utiliza modelos Pydantic, permitindo transformar a resposta do LLM em objetos com campos definidos.

Uso do LLM

O LLM é utilizado principalmente para interpretar conteúdo visual e semiestruturado do documento. As etapas posteriores de validação, persistência e conciliação permanecem sob controle do código.

8. Modelos estruturados e validação dos dados

O arquivo `models.py` concentra as classes Pydantic usadas como contrato de dados. Entre elas estão `SQLGerado`, `ItemNotaFiscal`, `NotaFiscal` e os modelos associados ao quadro-resumo.

Modelo	Função
QuadroResumo	Representar os dados principais extraídos do relatório.
ItemQuadroResumo	Representar uma despesa do quadro-resumo.
NotaFiscal	Representar um comprovante e seus dados principais.

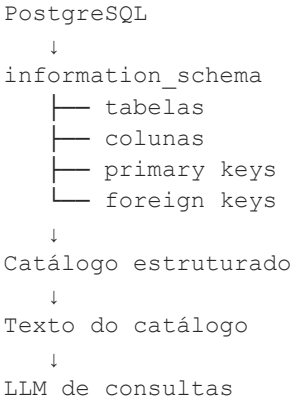
ItemNotaFiscal	Representar produtos/serviços detalhados do comprovante.
SQLGerado	Estruturar SQL, explicação e tabelas utilizadas.

Essa estratégia reduz a dependência de interpretação manual do texto retornado pelo modelo e cria uma interface estruturada entre o LLM e o restante da aplicação.

9. Banco de dados, catálogo e persistência

A camada de dados utiliza PostgreSQL/Supabase. O módulo `database.py` possui funções para obter tabelas, colunas, chaves primárias e chaves estrangeiras a partir do `information_schema`.

9.1 Construção automática do catálogo



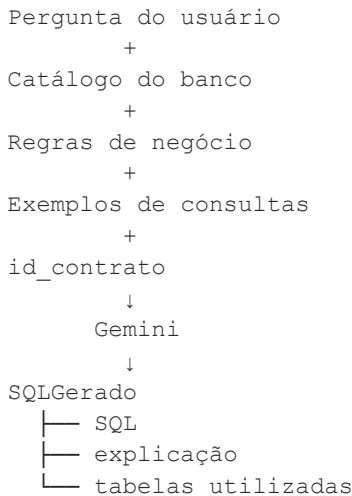
O catálogo representa o esquema disponível para a geração de SQL. O código também incorpora regras de negócio e exemplos de perguntas/consultas no contexto enviado ao modelo.

9.2 Persistência

O salvamento do relatório utiliza transação explícita. O código insere o relatório e, quando existentes, as despesas, comprovantes e respectivos itens. A transação é confirmada somente após a conclusão das etapas.

10. Consultas em linguagem natural: Text-to-SQL

A terceira opção da aplicação permite ao usuário formular perguntas em linguagem natural. O sistema constrói o catálogo do banco e fornece ao Gemini um contexto composto por catálogo, regras de negócio, exemplos, `id_contrato` e pergunta do usuário.



10.1 Configuração do modelo

O arquivo `config.py` define `SQL_LLM_MODEL` para o modelo utilizado na geração das consultas. A configuração padrão presente no código é `'gemini-3.5-flash-lite'`, podendo o valor ser alterado por variável de ambiente.

10.2 Saída estruturada

A chamada ao Gemini utiliza `temperature=0`, `response_mime_type='application/json'` e `response_schema=SQLGerado`. Dessa forma, a aplicação espera uma resposta estruturada e válida.

11. Validação e segurança do SQL

O SQL gerado pelo LLM passa por uma etapa independente de validação antes da execução. A função `preparar_e_validar_sql` verifica se o comando foi informado, se começa com `SELECT` e se não contém comandos considerados proibidos.

Controle	Comportamento
SQL vazio	Bloqueia a execução.
Comando diferente de SELECT	Bloqueia a execução.
Comandos proibidos	Bloqueia a execução.
SQL aprovado	Permite o encaminhamento para execução.
Contrato	<code>id_contrato</code> é obrigatório no contexto da geração.

Governança do LLM

O modelo não é tratado como autoridade única para segurança. A aplicação aplica uma validação determinística após a geração do SQL e antes de executar a consulta.

12. Conciliação de despesas e comprovantes

A conciliação é realizada por código, e não delegada integralmente ao LLM. O sistema busca despesas pendentes e comprovantes ainda não associados e procura correspondências com base em atributos dos registros.

- Número do documento.
- Valor da despesa.
- Nome do emitente.
- Data da despesa/emissão.
- Similaridade textual, utilizando a biblioteca TheFuzz.

Essa separação é relevante na arquitetura: o LLM interpreta os documentos e as perguntas, enquanto a associação dos registros utiliza regras computacionais explícitas.

13. Geração do relatório consolidado

A opção 2 utiliza report.py para consultar os dados e gerar um relatório consolidado em PDF com ReportLab. O módulo contempla tabelas, gráficos, cabeçalho, rodapé, paginação e formatação visual.

- Totais e consolidações de despesas.
- Despesas por categoria.
- Despesas por período.
- Gráficos de barras e pizza.
- Paginação e identificação do relatório.
- Arquivo PDF salvo para utilização posterior.

14. Estrutura do código

Arquivo	Responsabilidade principal
__init__.py	Identificação do pacote Python.
config.py	Configuração, credenciais, modelos e regras de negócio.
consultas.py	Text-to-SQL, validação e execução.
database.py	Banco, catálogo, persistência e conciliação.

main.py	Interface e orquestração.
models.py	Modelos Pydantic e contratos de dados.
processing.py	PDF, normalização, datas e extração multimodal.
report.py	Relatório consolidado em PDF.

15. Exemplos de execução

15.1 Processamento de relatórios

```
Digite o código da obra/contrato ou 0 - Sair: CONTRATO_X
```

```
Escolha uma opção:
```

- ```
1 - Processar relatório(s) de despesas
2 - Gerar relatório consolidado
3 - Consultar dados (Use linguagem natural)
0 - Sair
```

```
Digite a opção desejada: 1
```

Na opção 1, os PDFs selecionados são carregados, o sistema verifica se cada relatório já foi processado, envia o documento para extração, valida a correspondência do cliente e salva os dados.

### 15.2 Consulta em linguagem natural

```
Digite sua consulta aos dados em linguagem natural:
```

```
> Quais são as despesas pendentes de comprovação?
```

```
↓
```

```
Gemini
```

```
↓
```

```
SQLGerado
```

```
↓
```

```
Validação
```

```
↓
```

```
PostgreSQL/Supabase
```

```
↓
```

```
DataFrame com resultado
```

### 15.3 Geração de relatório

Na opção 2, o pipeline de relatório consulta os dados do contrato selecionado e gera um PDF consolidado com tabelas e visualizações.

# 16. Configuração e execução do projeto

As configurações sensíveis são carregadas por variáveis de ambiente a partir do arquivo .env. O código evita imprimir as credenciais na função de validação.

## 16.1 Variáveis relevantes

| Variável                                            | Finalidade                               |
|-----------------------------------------------------|------------------------------------------|
| SUPABASE_URL                                        | URL do projeto Supabase.                 |
| SUPABASE_KEY                                        | Chave de acesso do Supabase.             |
| SUPABASE_DB_URI                                     | URI de conexão do banco.                 |
| CHAVE_API_GEMINI                                    | Credencial utilizada pelo SDK Gemini.    |
| DB_NAME / DB_USER / DB_PASSWORD / DB_HOST / DB_PORT | Parâmetros de conexão PostgreSQL.        |
| SQL_LLM_MODEL                                       | Modelo utilizado para geração de SQL.    |
| EXTRACT_LLM_MODEL                                   | Modelo utilizado para extração dos PDFs. |

# 17. Tecnologias utilizadas

| Tecnologia/Biblioteca | Uso no projeto                           |
|-----------------------|------------------------------------------|
| Python                | Linguagem principal.                     |
| google-genai          | Integração com modelos Gemini.           |
| Pydantic              | Validação e estruturação das respostas.  |
| PostgreSQL            | Banco de dados relacional.               |
| Supabase              | Serviço de banco e acesso aos dados.     |
| psycopg2              | Conexão e execução de SQL no PostgreSQL. |
| Pandas                | Tratamento dos resultados das consultas. |
| TheFuzz               | Similaridade textual para conciliação.   |
| ReportLab             | Geração dos relatórios PDF.              |
| python-dotenv         | Carregamento das variáveis de ambiente.  |

# 18. Comparação entre proposta e implementação

A proposta original descrevia uma solução com automação da consolidação e auditoria, utilização de banco relacional, interação em linguagem natural e aplicação de técnicas de IA/LLM. A implementação atual concretiza parte importante desses objetivos, com uma arquitetura modular.

| Tema da proposta                      | Implementação observada                                                                                                                                                 |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Automação da consolidação de despesas | Implementada por persistência e geração de relatório consolidado.                                                                                                       |
| Processamento de PDFs                 | Implementado com extração via Gemini.                                                                                                                                   |
| Banco relacional                      | Implementado em PostgreSQL/Supabase.                                                                                                                                    |
| Consulta em linguagem natural         | Implementada com geração de SQL via Gemini.                                                                                                                             |
| RAG                                   | A proposta menciona RAG; a implementação atual está mais precisamente caracterizada como Text-to-SQL orientado por catálogo, regras e exemplos.                         |
| Múltiplos agentes especialistas       | A proposta apresenta essa arquitetura; o código enviado apresenta uma arquitetura modular, sem evidência suficiente para caracterizar o sistema atual como multiagente. |

## 19. Referências e links

### Repositório do projeto

<https://github.com/KarineVersieux/BIMASTER-TCC-projeto-gestao-despesas>

### Fontes e referências utilizadas no desenvolvimento e na documentação

#### Fontes internas do projeto:

- Proposta de Trabalho de Conclusão de Curso — *Automação dos processos de consolidação e auditoria de dados de custos*.
- Código-fonte do projeto: `main.py`, `config.py`, `processing.py`, `models.py`, `database.py`, `consultas.py` e `report.py`.
- Repositório do projeto no GitHub: **BIMASTER-TCC-projeto-gestao-despesas**, contendo o código-fonte, a documentação, exemplos e demais arquivos relacionados à implementação.

#### Documentação técnica e referências externas:

- **GOOGLE**. *Gemini API — Document understanding*. Documentação oficial sobre processamento e interpretação de documentos por modelos Gemini.

- **GOOGLE.** *Gemini API — Structured outputs*. Documentação oficial sobre a geração de respostas estruturadas utilizando modelos Gemini.
- **PYTHON SOFTWARE FOUNDATION.** *Documentação Python 3*. Documentação oficial da linguagem Python, incluindo referência da linguagem e da biblioteca padrão.
- **OPENAI.** *ChatGPT*. Ferramenta de inteligência artificial utilizada como apoio à **elaboração, revisão, depuração e documentação do código e dos materiais técnicos do projeto**.